tPC-RTRL을 활용한 시계열 실시간 학습 및 예측

이 석 원

강원대학교 산업대학원(강릉) 컴퓨터공학과

tPC-RTRL은 RG-LRU 구조를 기반으로 tPC와 RTRL을 결합함으로써 공간적, 시간적 국소성을 만족한다. 추론이 완전히 수렴할 경우, tPC-RTRL은 BPTT와 수학적으로 등가이다. 본 논문에서는 실시간 학습 환경에서 추론의 수렴을 강제하지 않고 tPC-RTRL이 역전파(BP) 기반 학습과 동일한 성능을 내는지 실험한다. Industrial Robot Identification Benchmark 데이터 세트를 사용하였다. tPC-RTRL은 tPC 대비 예측 오차(MSE)를 약 39% 감소시켰으며, BP 기반 변형 모델과는 0.03%의 성능 차이만을 보였다. TBPTT(k=5)로 학습한 GRU와 비교했을 때, tPC-RTRL은 MSE가 11.89% 더 높았으나, 메모리 사용량은 92.86%, 실행 시간은 64.51% 더 적게 소요되었다.

□ 핵심주제어

자유에너지 원리, 예측코딩, 실시간 순환 학습, 온라인 학습, 시계열 예측

* 심사가 종료된 날로 하므로, 1학기 제출자는 6월, 2학기 제출자는 12월이 됨

목 차

Ⅰ. 연구 배경 및 목적 ····· 1
1. 연구 배경 ····· 1
2. 본 연구의 주요 기여 ····· 2
3. 논문의 구성 ····· 3
Ⅱ. 관련 연구 ····· 5
1. 자유에너지 원리와 PC ····· 5
1) 베이지안 정리의 모델 증거 ····· 5
2) 변분 자유에너지 ····· 9
3) 예측 코딩 네트워크 ····· 11
4) tPC ····· 14
2. 순환 신경망의 학습 방법 ····· 16
Ⅲ. 모델 구현 ····· 18
1. tPC-RTRL ····· 18
1) 자유에너지 ····· 18
2) 추론 ····· 19
3) 가중치 학습 ····· 20
4) 순환부 RG-LRU ····· 21
5) 영향 행렬과 순환 가중치 학습 ····· 22
2. tPC-RTRL(BP) ····· 22
3. GRU ····· 23
4. 필터링 ····· 24
5. 제어 신호 추론 ····· 24
1) tPC-RTRL의 제어 신호 추론 ····· 29
2) GRU의 제어신호 추론 ····· 30

목 차

Ⅳ. 실험 결과 및 결과 분석 ····· 33
1. 데이터 세트 ····· 33
1) 가공 데이터를 사용하지 않은 이유 ····· 33
2) 관측 데이터와 제어 신호 ····· 34
3) 데시메이션 ····· 34
4) 엘리어싱과 저역 통과 필터 ····· 35
5) 정규화 ····· 36
2. 실험 절차와 초매개변수 ····· 36
1) 실험 절차 ····· 36
2) 초매개변수 ····· 37
3. 실시간 학습 및 예측 성능 ····· 39
4. 필터링 성능 ····· 40
5. 제어 신호 추론 성능 ····· 41
6. 비용 ····· 42
Ⅳ. 결론 ····· 46
1. 향후 연구 ····· 47
참고문헌 ····· 48

표 목 차

[표 1] ····· 6
[표 2] ····· 7
[표 3] ····· 37

그 림 목 차

[그림 1] ····· 7
[그림 2] ····· 9
[그림 3] ····· 10
[그림 4] ····· 14
[그림 5] ····· 25
[그림 6] ····· 38
[그림 7] ····· 39
[그림 8] ····· 40
[그림 9] ····· 41
[그림 10] ····· 43
[그림 11] ····· 44
[그림 12] ····· 45

Ⅰ. 연구 배경 및 목적

1.1 연구배경

오차 역전파 기반의 인공 신경망 알고리즘은[1] 뇌가 실제로 배우는 방식과 다르다는 지적을 오래전부터 받아왔다[2]. 역전파에서는 출력층에서 계산한 오차를 가장 깊은 층까지 거꾸로 보내야 한다. 또 이렇게 전달된 오차는 뉴런의 활동에는 영향을 주지 않고, 가중치를 고치는데만 쓰인다[3]. 그러나 뇌의 시냅스는 주변 뉴런에서 얻을 수 있는 정보만으로 변한다[4]. 이처럼 이웃한 뉴런끼리 주고받는 정보만으로 학습하는 성질을 공간적 국소성이라고 한다[5]. 한편, 공간적 국소성을 만족하면서 오차 역전파와 비슷한 학습을 할 수 있다는 연구도 있다. 예측 코딩(Predictive Coding, PC)은 그 대표적인 예이다[3, 5].

시계열 데이터를 다룰 때는 시간 축에서도 같은 문제가 생긴다. 순환 신경망을 학습시키는 방법인 BPTT(Backpropagation Through Time)는 과거 숨겨진 상태를 모두 저장해 두었다가, 시간을 거슬러 오차를 전파한다. 이 계산을 위해 메모리 요구량이 시퀀스 길이 T에 비례하여 O(T)로 증가한다[6]. 반면에 시간적 국소성은 과거 기록을 저장하거나 시간을 거슬러 계산하지 않고, 시간 순방향으로 유지되는 정보만으로 학습하는 성질을 갖는다[7].

tPC-RTRL은 이 두 가지 국소성을 함께 갖추도록 제안된 방법이다[7]. PC를 시계열로 확장한 tPC(Temporal Predictive Coding)는[8] 공간적 국소성을 지키지만, 장기 의존성을 학습하기 어렵다는 한계가 있다. 여기에 실시간 순환 학습(Real-Time

Recurrent Learning, RTRL)을 결합하면, 과거를 저장하지 않고도 매 스텝 업데이트하는 영향 행렬로 먼 과거의 영향까지 반영할 수 있다. H가 순환 숨겨진 상태의 차원, I가 제어 신호의 차원이라고 할때 순환 숨겨진 상태가 완전 연결된 일반적인 RTRL은 $O(H^3)$의 메모리가 필요하지만[9], 순환 구조가 단순한 RG-LRU[10]를 결합하면 O(HI)수준으로 줄일 수 있다[7].

tPC-RTRL은 추론의 완전 수렴 조건에서 BPTT와 수학적으로 등가임을 이론적으로 입증하였다[7]. 오프라인 배치 학습 기반의 나노드론 실험에서 tPC-RTRL의 추론의 완전한 수렴을 강제하지 않았는데도 BPTT와 비슷한 성능을 보였다[7]. 본 논문에서는 과연 실시간 학습에서도 tPC-RTRL이 BPTT와 비슷한 성능을 보이는지 실험한다.

1.2 본 연구의 주요 기여

본 연구는 Industrial Robot Identification Benchmark 데이터세트[11,12]를 사용하여, 세 가지 과제를 평가하였다. 워밍업 구간을 제외하고 데이터를 한 스텝씩 받으면서 예측 후 학습하는 실시간 시뮬레이션 환경을 구성한 실시간 학습 및 예측 과제, 온라인 학습을 마친 후 관측이 띄엄띄엄 주어질 때 숨겨진 상태를 추론하며 관측을 예측하는 필터링 과제, 온라인 학습을 마친 후 미래의 목표 관측을 만들어 내는 제어 신호를 찾는 과제이다.

비교 모델은 시공간 국소성을 모두 지키는 tPC-RTRL, 여기서 RTRL을 뺀 tPC, 그리고 두 모델에서 각각 공간 국소성을 완화한 BP 변형 모델 tPC(BP), tPC-RTRL(BP)이다. 이 모델들은 모두 같은 RG-LRU 순환 구조를 사용하므로, 성능 차이를 국소성 조건의 효과로 해석할 수 있다. 이와 별도로, 시계열 학습에서 널리 쓰이는

TBPTT(Truncated Backpropagation Through Time)[13]로 학습하는 GRU[14]를 기준선으로 함께 비교하였다. GRU는 순환 구조와 학습 방식이 모두 tPC-RTRL과 다르다. 따라서 GRU와의 성능 및 비용 차이는 순환 구조와 학습 방식이 함께 만든 결과로 해석해야 한다.

실시간 학습 및 예측에서 tPC-RTRL은 공간 국소성을 완화한 tPC-RTRL(BP)와의 예측 오차가 0.03%의 차이이며, 거의 같은 성능을 보였다. GRU와 비교하면 예측 오차는 약 11.89% 컸지만, 메모리는 약 92.86%, 실행 시간은 약 64.51% 적게 사용하였다. 능동 추론과 PC에 익숙하지 않은 독자를 위해, 베이즈 정리부터 tPC까지 필요한 배경을 단계적으로 정리하였다.

1.3 논문의 구성

본 논문의 구성은 다음과 같다. 제 2 장에서는 tPC-RTRL을 이해하는 데 필요한 배경을 정리한다. 2.1 절에서는 베이즈 정리의 모델 증거에서 출발한다. 이어서 변분 자유에너지와 예측 코딩 네트워크(PCN)를 설명하고, 이를 시계열로 확장한 tPC까지 단계적으로 다룬다. 2.2 절에서는 순환 신경망의 학습 방법인 BPTT, TBPTT, RTRL을 시간적 국소성과 메모리 비용 관점에서 비교한다. 또한 RG-LRU의 원소별 순환 구조가 근사 없이 정확한 RTRL을 가능하게 하는 이유를 설명한다.

제 3 장에서는 실험에 사용한 모델과 과제를 구현 수준에서 기술한다. 3.1 절에서는 tPC-RTRL의 자유에너지, 추론, 가중치 학습, RG-LRU 순환부, 그리고 영향 행렬을 이용한 순환 가중치 학습을 차례로 설명한다. 3.2 절에서는 공간적 국소성을 완화한 BP 변형 모델을 설명하고, 3.3 절에서는 기준선으로 사용한 TBPTT 기반 GRU를 설명한다. 3.4 절과 3.5 절에서는 같은 생성 모델로 수행하는

필터링과 제어 신호 추론을 설명하고, 제어 신호에 대한 기울기를 모델별로 유도한다.

제 4 장에서는 실험 결과를 분석한다. 4.1 절에서는 Industrial Robot Identification Benchmark 데이터세트와 전처리 과정을 설명한다. 4.2 절에서는 실험 절차와 초매개변수를 기술하고, tPC-RTRL과 tPC-RTRL(BP)의 기울기를 비교하여 본 실험이 근사 등가 영역에서 수행됨을 확인한다. 4.3 절부터 4.5 절까지는 실시간 학습 및 예측, 필터링, 제어 신호 추론 성능을 차례로 비교한다. 4.6 절에서는 메모리 사용량과 타임 스텝당 실행 시간을 비교한다.

제 5 장에서는 연구 결과를 요약하고, 향후 연구로 주의와 정서가의 기능을 tPC-RTRL 기반 실시간 시스템에 구현하는 방향을 제시한다.

Ⅱ. 관련 연구

2.1 자유에너지 원리와 PC

자유에너지 원리(Free-Energy Principle)는 뇌가 외부 세계에 대한 내부 생성 모델을 바탕으로 감각 입력을 예측하고, 감각의 놀람(surprise)에 대한 상한인 자유에너지를 최소화하는 방향으로 지각과 행동을 수행한다는 이론이다[15]. 이 원리에서 지각은 내부 상태를 업데이트하여 감각 입력의 원인을 추론하는 과정이고, 행동은 예측에 부합하는 감각 입력을 얻도록 환경에 작용하는 과정이다. Friston은 후자를 능동 추론(Active Inference)이라 불렀으며[15], 이후 능동 추론은 지각과 행동을 자유에너지 최소화로 함께 설명하는 틀로 정리되었다[16]. 정규 분포 가정 아래에서 자유에너지 최소화는 예측 오차 최소화와 대응되며, 이를 지각 측면에서 구현하는 알고리즘이 예측 코딩(Predictive Coding, PC)이다[15]. PC는 최근 머신러닝 분야에서도 연구되고 있다. 특정 조건에서 PC가 오차 역전파와 같은 기울기를 계산한다는 것이 밝혀졌다[17].

2.1.1 베이지안 정리의 모델 증거

자유에너지 원리는 베이지안 정리에서 출발한다. 책상 위에 뚜껑이 덮여있는 텀블러를 드는 단순한 상황을 생각해보자. 우리는 텀블러 안에 커피가 얼마나 들어있는지(숨겨진 상태 x)를 직접 볼 수 없고, 컵을 들 때 손과 팔에 전해지는 무게감(관찰 y)만 느낄 수 있다. 이 무게감으로부터 커피의 양을 추론하는 과정은 수식 (1)로 표현할 수

있으며 이를 사후 확률이라고 한다.

$$p(x|y) = \frac{p(y|x)p(x)}{p(y)} \tag{1}$$

분자의 $p(y|x)$는 커피가 x만큼 들어 있을 때 무게감 y가 느껴질 확률이고 가능도라고 한다. $p(x)$는 컵을 들기전의 예상이고 사전 확률이라고 한다. 분모의 $p(y)$ 커피의 양과 무관하게 이 무게감이 느껴질 총 확률, 즉 내가 가진 예상 전체에 비추어 볼때 이 감각이 얼마나 그럴듯한가를 나타내며 모델 증거라고 한다.

이해를 돕기 위해 커피의 양(x)을 '가득(Full)', '절반(Half)', '빈컵(Empty)'세가지로 단순화하고, 무게감(y)을 '무거움(Heavy)', '보통(Normal)', '가벼움(Light)'세가지로 단순화해 보자. 컵에 든 양에 따라 각 무게감이 느껴질 확률 $p(y|x)$은 표 1과 같다고 가정한다.

Table 1 The likelihood of heaviness depending on the amount of coffee $p(y|x)$
표 1 커피의 양에 따른 무게감의 확률 $p(y|x)$

	Heavy	Normal	Light
Full	0.95	0.04	0.01
Half	0.05	0.9	0.05
Empty	0.01	0.04	0.95

이제 실제로 '가벼운' 텀블러를 들때 두 상황을 비교해 보자. 상황 A는 방금 커피를 따랐으니 텀블러가 무겁다고 믿는 경우, 상황 B는 커피를 다 마셨으니 텀블러가 가볍다고 믿는 경우로, 그림 1에 상황을 표현했다. 각 상황에 따른 사전 확률은 표 2와 같다고 가정한다.

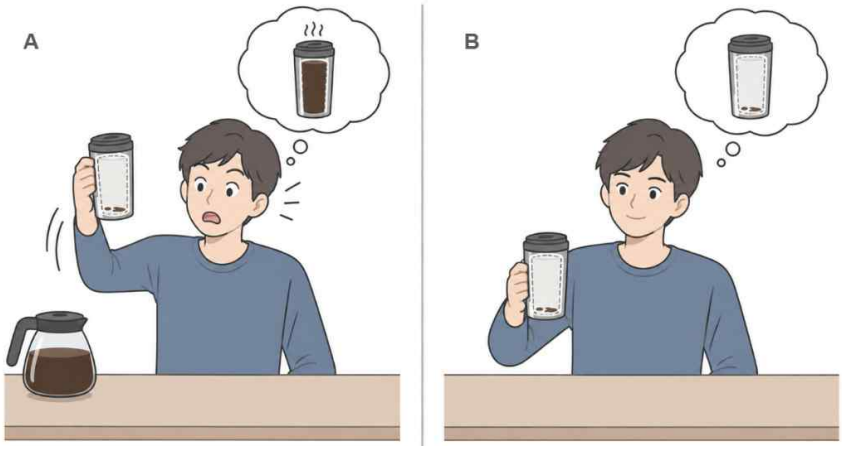


그림 1 텀블러가 무겁다고 믿고 있는 상황 A와 가볍다고 믿고 있는 상황 B를 나타낸 그림

Fig. 1. A picture showing the situation A where you believe the tumbler is heavy and the situation B where you believe it is light

Table 2 Prior probability depending on the situations $p(x)$
표 2 상황에 따른 사전 확률 $p(x)$

	Full	**Half**	**Empty**
상황 A	0.8	0.15	0.05
상황 B	0.05	0.15	0.8

이제 베이지안 정리에 따라 사후 확률을 계산해보자. 모델 증거는 수식 (2)를 통해 계산할 수 있다. 지금 텀블러는 실제로 '가볍다'고 가정 했으므로 상황 A, B의 모델 증거는 수식 (3-4)와 같고 사후 확률은 수식 (5-6)과 같다.

$$p(y_{Light}) = p(y_{Light}|Full)p(Full) + p(y_{Light}|Half)p(Half) + p(y_{Light}|Empty)p(Empty) \quad (2)$$

$$p_A(y_{Light}) = 0.01 \times 0.8 + 0.05 \times 0.15 + 0.95 \times 0.05 = 0.063 \quad (3)$$

$$p_B(y_{Light}) = 0.01 \times 0.05 + 0.05 \times 0.15 + 0.95 \times 0.8 = 0.768 \quad (4)$$

$$p_A(x|y_{Light}) = \frac{(0.95 \times 0.05)}{0.063} \approx 0.75 \quad (5)$$

$$p_B(x|y_{Light}) = \frac{(0.8 \times 0.95)}{0.768} \approx 0.99 \quad (6)$$

이렇게 두 상황 모두 사후 확률은 텀블러가 비어있을 가능성이 크다는 결론을 내린다. 그러나 모델 증거는 0.063과 0.768로 약 열배 차이가 난다. 상황 A에서는 가득 찬 줄 알고 든 텀블러가 가벼워 팔이 생각보다 높이 들리며 흠칫 놀라지만, 상황 B에서는 예상대로의 감각이므로 아무렇지도 않다. 이 차이를 수치로 나타낸 것이 모델 증거의 음의 로그 $-\ln p(y)$, 즉 놀람(surprise)이다. 상황 A의 놀람은 $-\ln p(0.063) \approx 2.76$, 상황 B의 놀람은 $-\ln p(0.768) \approx 0.26$이다. 이처럼 사후 확률은 결론만 알려주지만, 모델 증거는 그 감각이 나의 예상과 얼마나 어긋났는지를 알려준다. 자유에너지 원리는 환경과 평형 상태에 있는 모든 자기 조직화 시스템은 놀람의 상한, 즉 자유에너지를 최소화해야 한다고 본다[15].

하지만 실제 커피의 양은 연속적인 값이고, 텀블러의 재질, 손의 피로도, 드는 각도 등 무게감에 영향을 주는 숨은 상태도 많다. 이를 모두 고려하려면 모델 증거는 수식 (7) 처럼 가능한 모든 상태에 대한 적분이 되며, 텀블러를 들 때마다 이를 정확히 계산하는 것은 굉장히 어렵다.

$$p(y) = \int p(y|x)p(x)\,dx \qquad (7)$$
$$= \int p(y,x)dx$$

2.1.2 변분 자유에너지

모델 증거를 구하기 위해 적분을 직접 계산하는 것이 아니라 Jensen의 부등식을 이용한 최적화 문제로 바꿔서 모델 증거의 근사값을 구할 수 있다[18].

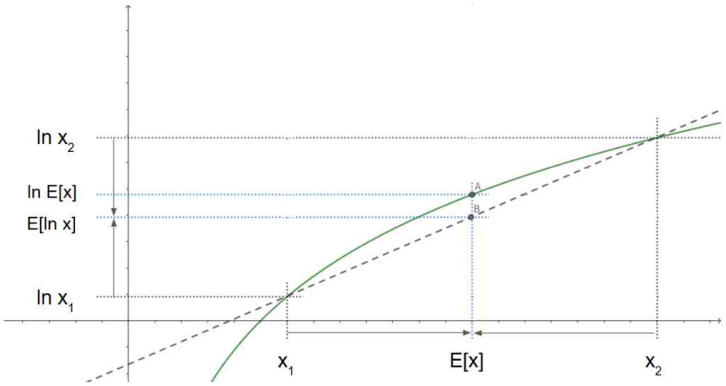


그림 2 로그 함수에서 Jensen의 부등식

Fig. 2. Jensen's inequality in the logarithm function

그림 2에서 볼 수 있듯이 Jensen의 부등식에 따르면 $\ln E[x] \geq E[\ln x]$가 성립한다. 임의의 확률 분포 함수 Q(x)를 이용하여 수식 (7)의 모델 증거를 기댓값 형태의 수식 (8)로 만들면 수식 (9)와 같이 Jensen의 부등식을 적용할 수 있다. 이때 수식 (9)의 우변을 음의 변분 자유에너지(-F)라고 하며 이는 로그 모델 증거 $\ln p(y)$의 하한이다. 이때 양변에 -1을 곱하면 수식 (10)과 같다. 따라서 자유에너지 F는

놀람의 상한이 된다.

$$\ln p(y) = \ln \int p(y,x)\frac{Q(x)}{Q(x)}dx = \ln E_{Q(x)}\left[\frac{p(y,x)}{Q(x)}\right] \tag{8}$$

$$\ln E_{Q(x)}\left[\frac{p(y,x)}{Q(x)}\right] \geq E_{Q(x)}\left[\ln \frac{p(y,x)}{Q(x)}\right] \doteq -F \tag{9}$$

$$F \geq -\ln p(y) \tag{10}$$

$$F = E_{Q(x)}[\ln Q(x) - \ln p(y,x)] \tag{11}$$

$$F = D_{KL}[Q(x)\|p(x|y)] - \ln p(y) \tag{12}$$

수식 (11)을 KL-분산으로 수식 (12)와 같이 표현할 수 있다. 이렇게 하면 변분 자유에너지는 뇌의 추정 Q(x)와 사후분포 $p(x|y)$가 얼마나 다른가와 데이터 y가 뇌의 모델 입장에서 얼마나 뜻밖인가로 해석할 수 있다.

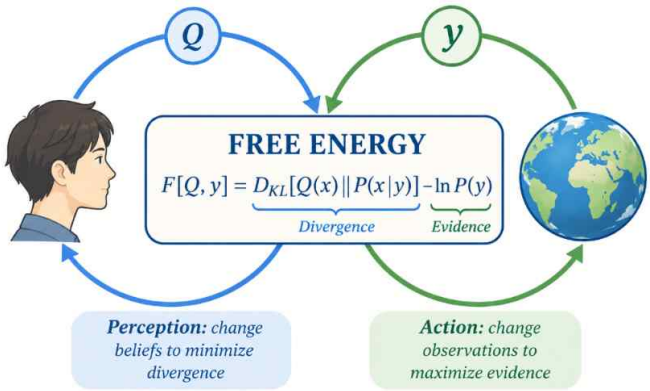


그림 3 변분 자유에너지를 최소화하는 두 경로: 지각과 행동[16]

Fig. 3. Two routes to minimizing variational free energy: perception and action [16]

수식 (12)를 그림 3에 나타내었다. 수식 (12)는 자유에너지를 낮추는 두 가지 경로를 보여 준다(그림 3). 첫째 항인 KL 발산은 뇌의 추정 Q(x)에 의존하지만, 둘째 항 $-\ln p(y)$는 Q(x)와 무관하다. 따라서 관측 y가 주어졌을 때 Q(x)를 바꾸어 F를 최소화하면 KL 발산만 줄어든다. 그 결과 Q(x)는 사후분포에 가까워진다.

F을 줄이기 위해 Q(x)뿐만 아니라 관측 y도 바꿀 수 있다. 이는 환경에 작용하는 행동을 통해서만 가능하다[15]. 텀블러의 예에서 상황 A의 사람은 '텀블러가 비어 있다'고 믿음을 고칠 수 있다(지각). 또는 커피를 따라 텀블러를 실제로 무겁게 만들어, 감각을 자신의 예상에 맞출 수도 있다(행동).

2.1.3 예측 코딩 네트워크(Predictive Coding Network, PCN)

수식 (11)에서 자유에너지를 최소화하기 위한 함수 Q(x)를 찾아야 한다. 이것을 디랙 델타 함수로 가정하면 수식을 간단하게 분해할 수 있다[19]. 디랙 델타 함수는 수식 (13-14)과 같은 성질이 있다.

$$\delta(x-\mu)=\begin{cases}\infty, x=\mu\\ 0, x\neq\mu\end{cases},\ \int\delta(x-\mu)dx=1 \tag{13}$$

$$\int Q(x)f(x)dx=\int\delta(x-\mu)f(x)dx=f(\mu) \tag{14}$$

확률이 μ한 점에 집중되어 있으므로 Q(x)의 형태는 μ하나로 결정된다. 따라서 Q(x)를 찾는 문제는 자유에너지를 최소화하는 μ를 찾는 문제로 바뀐다. 따라서 결합확률의 기댓값은 $\ln p(y,\mu)$가 된다. $E_Q[\ln Q(x)]$는 디랙 델타 함수에서 발산하지만 μ와는 무관하므로 상수 취급할 수 있다. 따라서 수식 (15)와 같이 가능도와 사전 확률로 분해되고 이것을 다시 정규 분포를 따른다고 하면 수식 (16)과 같이

방정식 형태가 된다.

$$F = -\ln p(y,\mu) + c = -(\ln p(y|\mu) + \ln p(\mu)) + c \tag{15}$$

$$F = \frac{(y-\theta g(\mu))^2}{2\Sigma_y} + \frac{1}{2}\ln\Sigma_y + \frac{(\mu-\eta)^2}{2\Sigma_x} + \frac{1}{2}\ln\Sigma_x + c \tag{16}$$

g는 비선형 함수, θ는 가중치, η는 사전 확률 분포의 평균, Σ는 분산이다. 여기서 F를 최소화하는 단계가 구분된다[5,17,19]. 첫 번째는 수식 (17-18)을 통해 F를 μ에 대해 미분하여 μ를 업데이트하는 것이고 이를 추론이라고 한다[5,17,19]. 본 논문에서는 α_{ir}를 추론률(Inference rate)라고 부른다.

$$\frac{\partial F}{\partial \mu} = \frac{(y-\theta g(\mu))\theta g'(\mu)}{\Sigma_y} + \frac{(\mu-\eta)}{\Sigma_x} \tag{17}$$

$$\mu \leftarrow \mu - \alpha_{ir}\frac{\partial F}{\partial \mu} \tag{18}$$

두 번째는 가중치와 분산을 업데이트하는 것이고 이를 학습이라고 한다[5,17,19]. 본 논문에서 α_{lr}은 학습률(Learning rate)이라고 부른다. 자유에너지를 최소화하는데 관측을 볼 때마다 추론을 수렴시킨 후 가중치와 분산은 수렴시키지 않고 조금씩 업데이트한다[19].

$$\frac{\partial F}{\partial \theta} = \frac{(y-\theta g(\mu))g(\mu)}{\Sigma_y} \tag{19}$$

$$\frac{\partial F}{\partial \Sigma_y} = -\frac{(y-\theta g(\mu))^2}{2\Sigma_y^2} + \frac{1}{2\Sigma_y} \tag{20}$$

$$\frac{\partial F}{\partial \Sigma_x} = -\frac{(\mu-\eta)^2}{2\Sigma_y^2} + \frac{1}{2\Sigma_x} \tag{21}$$

$$\theta \leftarrow \theta - \alpha_{lr} \frac{\partial F}{\partial \theta} \tag{22}$$

$$\Sigma_y \leftarrow \Sigma_y - \alpha_{lr} \frac{\partial F}{\partial \Sigma_y} \tag{23}$$

$$\Sigma_x \leftarrow \Sigma_x - \alpha_{lr} \frac{\partial F}{\partial \Sigma_x} \tag{24}$$

관측 y를 최하위 노드, 숨겨진 상태 μ를 중간 노드, η를 최상위 노드라고 하면, 수식 (17)의 추론 과정은 μ 이용한 예측과 관측 y의 오차와 자신의 μ와 상위 η의 예측 오차를 줄이는 방향으로 업데이트 된다. 수식 (19)의 가중치 업데이트는 함께 활동하는 뉴런은 서로 연결이 강해진다는 헤브 규칙(Hebbian rule)과 같은 형태로, 시냅스 양쪽 뉴런의 정보만으로 학습이 가능함을 뜻한다[19]. 이렇게 PCN는 추론과 학습 모두 인접한 노드간 값을 주고받으며 국소적으로 업데이트된다. 다만 분산의 경우 업데이트가 국소적이지 않은데 Bogaz(2017)[19]는 분산 업데이트를 국소화할 수 있는 인터 뉴런을 제안하였다. 하지만 tPC-RTRL에서는 분산을 학습시키지 않기 때문에 본 논문에서는 이에 대해서 다루지 않는다.

이 PCN는 벡터와 계층 구조 모두 확장 가능한데[19], 만약 수식 (16)에 계층을 하나 더 추가한다면 수식 (25)와 같고 각 오차 또한 노드라고 하고 이 계층 구조를 그림 4와 같이 나타낼 수 있다.

$$\begin{aligned} F = & \frac{1}{2}\Sigma_y^{-1} \| y - \theta_o g(o) \|^2 + \frac{1}{2}\ln\Sigma_y \\ & + \frac{1}{2}\Sigma_o^{-1} \| o - \theta_\mu g(\mu) \|^2 + \frac{1}{2}\ln\Sigma_o \\ & + \frac{1}{2}\Sigma_x^{-1} \| (\mu - \eta) \|^2 + \frac{1}{2}\ln\Sigma_x + c \end{aligned} \tag{25}$$

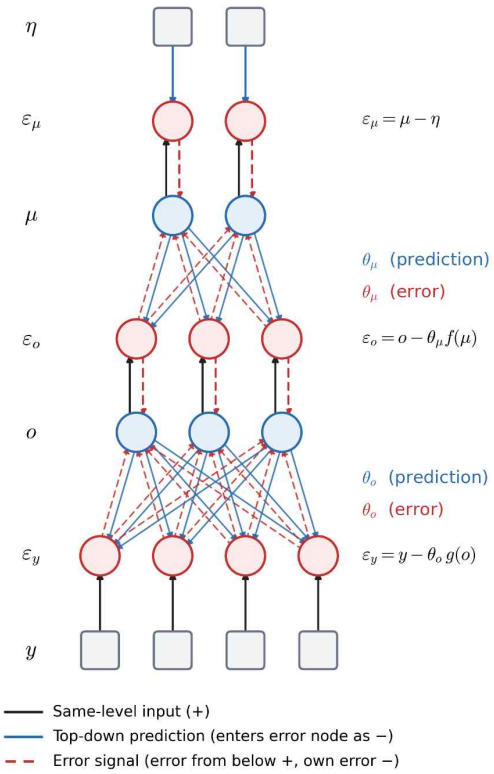


그림 4 PCN의 노드간 관계를 나타낸 그림
Fig. 4. A diagram showing the relationships between nodes in a PCN

2.1.4 tPC(Temporal Predictive Coding)

지금까지는 한순간의 숨겨진 상태 x만을 다루었다. 그러나 실제 감각 입력은 시간에 따라 들어오므로, PC가 시계열을 다루려면 시간축을 모델에 포함해야 한다.

이를 위한 기존의 대표적인 방법은 일반화된 좌표(generalized coordinates of motion)이다[18]. 이 방법은 테일러급수를 이용하여 숨겨진 상태의 값뿐 아니라 속도, 가속도 같은 시간 미분을 숨겨진 상태 벡터에 함께 넣고, 이 미분들까지 예측하도록 한다. 그러나 이를 신경 회로로 구현하려면 연결 구조가 복잡해지고 생물학적 타당성이 떨어진다[8].

tPC는 테일러급수를 사용하지 않고, 정적 PC의 단순한 구조를 그대로 유지하면서 시간 축을 도입하기 위해 제안되었다[8]. 대신 직전 스텝에서 추론한 상태 $\hat{x}^{(t-1)}$를 순환 연결로 다음 스텝에 전달하여 현재의 숨겨진 상태로 사용한다. 이때 상태 전이를 A, 제어 입력의 영향을 B, 관측 생성을 C와 비선형 함수 f로 나타내면, 자유에너지는 수식 (26)와 같은 방식으로 결합 확률의 음의 로그가 된다.

$$\begin{aligned} F^{(t)} &= -\ln p(y^{(t)}, x^{(t)} | \hat{x}^{(t-1)}, u^{(t)}) \\ &= -\ln p(y^{(t)} | x^{(t)}) - \ln p(x^{(t)} | \hat{x}^{(t-1)}, u^{(t)}) \end{aligned} \tag{26}$$

가능도와 사전 확률이 정규 분포를 따른다고 가정하고 숨겨진 상태 추정과 무관한 상수항을 생략하면, 수식 (26)는 수식 (27)이 된다.

$$\begin{aligned} F^{(t)} = &\frac{1}{2} \| y^{(t)} - Cf(x^{(t)}) \|^2 \Sigma_y^{-1} \\ &+ \frac{1}{2} \| x^{(t)} - Af(\hat{x}^{(t-1)}) - Bu^{(t)} \|^2 \Sigma_x^{-1} \end{aligned} \tag{27}$$

칼만 필터는 A, C를 알고 있어야 하지만, tPC는 이를 학습할 수 있다. 이렇게 학습한 모델의 관측 예측 성능은 칼만 필터와 비슷한 성능을 보였다. 하지만 tPC는 장기 의존성을 학습하기 어렵다는 한계가 있다[7,8]. tPC-RTRL은 이 한계를 RTRL로 보완했고, 추론의

완전 수렴시 BPTT와 등가임을 이론적으로 증명했다[7].

2.2 순환 신경망의 학습 방법

순환 신경망을 학습하는 대표적인 방법은 BPTT이다[6]. BPTT는 과거의 숨겨진 상태를 모두 저장한 뒤 시간을 거슬러 기울기를 계산한다. 그래서 메모리가 시퀀스 길이에 비례하고, 시퀀스가 끝나야 가중치를 갱신할 수 있다. TBPTT는 역전파 구간을 최근 k 시점으로 제한하여 이 문제를 줄인다[13]. 대신 k보다 오래된 영향은 반영하지 못하고, 온라인 학습에서는 k 스텝의 역전파를 반복하므로 계산량이 k에 비례하여 커진다.

RTRL은 과거 상태를 저장하지 않는다[9]. 대신 순환 가중치가 현재 숨겨진 상태에 미친 누적 영향을 행렬 $M^{(t)} = dx^{(t)}/d\theta$ 로 유지하고, 이를 수식 (28)과 같이 매 스텝 순방향으로 갱신한다.

$$M^{(t)} = \frac{\partial x^{(t)}}{\partial x^{(t-1)}} M^{(t-1)} + \frac{\partial x^{(t)}}{\partial \theta} \tag{28}$$

따라서 매 시점 BPTT와 같은 기울기를 얻으면서 시간적 국소성을 만족한다. 다만 모든 유닛이 서로 연결된 밀집 순환층에서는 영향 행렬에 $O(N^3)$의 메모리가 필요하다[9].

RG-LRU는 각 순환 숨겨진 상태의 원소가 직전 스텝의 자기 자신에게만 연결되는 원소별 대각 순환 구조이고, 게이트도 현재 입력에만 의존한다. 3.1.4 순환부 RG-LRU절을 보면 알 수 있다. 그래서 각 가중치는 자기 유닛에만 영향을 주고, 영향 행렬의 크기가 순환 숨겨진 상태의 가중치 수 수준으로 줄어 근사 없이 정확한 RTRL을 수행할 수 있다[7]. 반면 3.3절의 GRU 수식을 보면 GRU는

게이트에 직전 숨겨진 상태가 행렬 곱으로 들어가므로 이렇게 메모리 비용의 이점이 있는 구조를 쓸 수 없다. 본 논문에서 GRU를 TBPTT로 학습하는 이유이다.

tPC-RTRL은 tPC의 순환부에 RG-LRU[10]를 사용하고 순환 가중치 갱신에 영향 행렬을 통한 과거 영향 항을 더하여 tPC의 장기 의존성 한계를 보완한다[7]. 추론은 tPC와 같으므로 공간적 국소성을, 영향 행렬은 순방향으로만 갱신되므로 시간적 국소성을 함께 만족한다[7].

Ⅲ. 모델 구현

3.1 tPC-RTRL

3.1.1 자유에너지

tPC-RTRL의 구조는 기존 논문에서 제시한 구조를 그대로 따른다[7]. 모델은 크게 제어 신호 $u^{(t)} \in \mathbb{R}^I$, 순환 숨겨진 상태 $x^{(t)} \in \mathbb{R}^H$, 중간 숨겨진 상태 $o^{(t)} \in \mathbb{R}^R$, 관측 데이터 $y^{(t)} \in \mathbb{R}^C$로 구성된다. I는 제어 신호의 채널 수, H는 순환 숨겨진 상태의 차원, R은 중간 숨겨진 상태의 차원, C는 관측 데이터 입력 센서의 채널 수이다. 이는 2.1.4 그림 3의 3단 구조와 같다. 다만 순환 숨겨진 상태 $x^{(t)}$의 사전 평균은 2.1.5의 tPC처럼 직전 시점의 추론 결과와 현재 제어 신호로부터 순환부가 만든다. 이는 수식 (29)와 같으며, 순환부 구조는 3.1.4에서 설명한다.

$$\mu_x^{(t)} = f_\theta(\bar{x}^{(t-1)}, u^{(t)}) \tag{29}$$

분산은 학습시키지 않으며, 각 항을 해당 계층의 차원 수로 정규화하면 스텝당 자유에너지는 수식 (30)과 같다. ϕ는 ReLU 함수이다.

$$F^{(t)} = \frac{1}{2H} \| x^{(t)} - \mu_x^{(t)} \|^2 + \frac{1}{2R} \| o^{(t)} - \phi(W_r x^{(t)} + b_r) \|^2 + \frac{1}{2C} \| y^{(t)} - (W_l o^{(t)} + b_l) \|^2 \quad (30)$$

3.1.2 추론

매 시점 관측 $y^{(t)}$가 들어오면, 가중치를 고정한 채 수식 (30)을 $x^{(t)}$와 $o^{(t)}$에 대해 경사하강으로 최소화한다. 기울기는 수식 (32-33)과 같으며, $m_r^{(t)}$는 ReLU의 미분이다.

$$m_r^{(t)} = \begin{cases} 1 \text{ if } W_r \mu_x^{(t+\tau)} + b_r > 0 \\ 0 \; otherwise \end{cases} \quad (31)$$

$$\frac{\partial F^{(t)}}{\partial x^{(t)}} = \frac{1}{H}(x^{(t)} - \mu_x^{(t)}) - \frac{1}{R} W_r^T((o^{(t)} - \phi(W_r x^{(t)} + b_r)) \odot m_r^{(t)}) \quad (32)$$

$$\frac{\partial F^{(t)}}{\partial o^{(t)}} = \frac{1}{R}(o^{(t)} - \phi(W_r x^{(t)} + b_r)) - \frac{1}{C} W_l^T(y^{(t)} - (W_l o^{(t)} + b_l)) \quad (33)$$

각 상태는 자신의 오차와 바로 아래 계층의 오차만으로 업데이트된다. 이 수식 (34)를 정해진 횟수만큼 반복하며, 업데이트에는 모멘텀을 적용한다[7].

$$x^{(t)} \leftarrow x^{(t)} - \alpha_{lr} \frac{\partial F^{(t)}}{\partial x^{(t)}}, \; o^{(t)} \leftarrow o^{(t)} - \alpha_{lr} \frac{\partial F^{(t)}}{\partial o^{(t)}} \quad (34)$$

추론을 마친 $x^{(t)}$는 가중치 업데이트에 필요한 예측 오차를

계산하는 데에만 쓰이고, 다음 시점으로는 $\bar{x}^{(t)} \leftarrow \mu_x$에 따라 순환부의 예측을 넘긴다[7].

3.1.3 가중치 학습

가중치는 원논문과 같이 순방향 업데이트 방식이다[7]. 이 방식에서는 예측값과 기울기 계산에 쓰이는 입력을 모두 추론 전의 순방향 값 $x^{(t,0)} = \mu_x^{(t,0)}$, $o^{(t,0)} = \phi(W_r x^{(t,0)} + b_r)$로 계산한다. 추론을 마친 $\hat{o}^{(t)}$는 예측과의 차이를 구할 때만 사용한다. 그 결과 테스트 때 실제로 쓰는 순방향 경로를 기준으로 가중치가 업데이트되므로, 학습과 예측 사이의 불일치가 줄고 학습이 안정된다[7]. 출력부 가중치의 기울기는 수식 (35-36)과 같다. 여기서 $m_r^{(t)}$ 은 수식 (31)을 이용한다.

$$\frac{\partial F^{(t)}}{\partial W_l} = -\frac{1}{C}(y^{(t)} - (W_l o^{(t,0)} + b_l))\hat{o}^{(t)T}, \tag{35}$$
$$\frac{\partial F^{(t)}}{\partial b_l} = -\frac{1}{C}(y^{(t)} - (W_l o^{(t,0)} + b_l))$$

$$\frac{\partial F^{(t)}}{\partial W_r} = -\frac{1}{R}((\hat{o}^{(t)} - o^{(t,0)}) \odot m_r^{(t)})x^{(t,0)T}, \tag{36}$$
$$\frac{\partial F^{(t)}}{\partial b_r} = -\frac{1}{R}(\hat{o}^{(t)} - o^{(t,0)}) \odot m_r^{(t)}$$

가중치는 매 시점 이 기울기로 수식 (37)과 같이 갱신하며, 실제 갱신에는 Adam을 사용한다[7].

$$W \leftarrow W - \alpha_{lr} \frac{\partial F^{(t)}}{\partial W} \tag{37}$$

3.1.4 순환부 RG-LRU

$\mu_x^{(t)}$를 만드는 순환부 수식 (29)로는 원 논문과 같이 RG-LRU[10]을 사용한다[7]. 순환부의 계산은 수식 (38-43)과 같다.

$$g_a^{(t)} = \sigma(W_a u^{(t)} + b_a) \tag{38}$$

$$g_z^{(t)} = \sigma(W_z u^{(t)} + b_z) \tag{39}$$

$$p^{(t)} = W_{in} u^{(t)} + b_{in} \tag{40}$$

$$\log a^{(t)} = c g_a^{(t)} \odot \log \sigma(\Lambda) \tag{41}$$

$$\gamma^{(t)} = \sqrt{1 - \exp(2 \log a^{(t)})} \tag{42}$$

$$\mu_x^{(t)} = a^{(t)} \odot \bar{x}^{(t-1)} + \gamma^{(t)} \odot (g_z^{(t)} \odot p^{(t)}) \tag{43}$$

σ는 시그모이드 함수, c는 고정된 상수이다. $\mu_x^{(t)}$는 순환 숨겨진 상태를 예측한 값이다. 순환 가중치는 $\theta_{rec} = \{\Lambda, W_a, b_a, W_z, b_z, W_{in}, b_{in}\}$ 이며, $W_a, W_z, W_{in} \in \mathbb{R}^{H \times I}$, $b_a, b_z, b_{in} \in \mathbb{R}^H$이다.

RG-LRU의 핵심은 수식 (43)이 원소별 연산이라는 점이다. $\mu_x^{(t)}$의 각 원소는 $\bar{x}^{(t-1)}$의 같은 원소에만 의존하고, 게이트는 현재 입력 $u^{(t)}$에만 의존한다. 따라서 영향 행렬의 메모리는 O(HI) 수준이 된다. 수식 (40)는 제어 입력의 차원 I와 순환 잠재 상태의 차원 H를 맞추기 위해 필요하다.

3.1.5 영향 행렬과 순환 가중치 학습

2.2절의 RTRL 수식 (28)을 RG-LRU에 적용하면 순환 가중치 $\theta_{rec} = \{\Lambda, W_a, b_a, W_z, b_z, W_{in}, b_{in}\}$에 대한 영향 행렬 업데이트식은 다음 수식 (44)와 같다.

$$M_\theta^{(t)} = a^{(t)} \odot M_\theta^{(t-1)} + \frac{\partial \mu_x^{(t)}}{\partial \theta_{rec}^{(t)}} \tag{44}$$

자유에너지 수식 (30)에서 θ_{rec}는 첫째 항의 $\mu_x^{(t)}$를 통해서만 영향을 준다. 따라서 추론을 마친 $\hat{x}^{(t)}$를 이용하면 순환 가중치의 기울기와 업데이트는 수식 (45-46)와 같으며 Adam을 사용한다[7]. 수식 (44)에서 첫째 항을 빼고 직접 영향만 쓰면 표준 tPC의 순환 가중치 학습이 된다.

$$\nabla \theta_{rec} = -\frac{1}{H}(x^{(t)} - \mu_x^{(t)}) M_{\theta_{rec}} \tag{45}$$

$$\theta_{rec} \leftarrow \theta_{rec} - \alpha_{lr} \nabla \theta_{rec} \tag{46}$$

3.2 tPC-RTRL(BP)

공간 국소성 조건을 완화하기 위해 추론 단계에서 중간 숨겨진 상태 $o^{(t)}$의 수렴 절차를 건너뛰고 관측 예측 오차$(y^{(t)} - (W_l o^{(t)} + b_l))$를 $x^{(t)}$까지 직접 오차 역전파 형태로 전달하여 $x^{(t)}$의 변화량만을 구한다. 따라서 반복 추론을 통해 오차를 구하지 않는다. 필터링 실험에서는 가중치를 고정하고 수식 (47)을

$x^{(t)}$에 대해 경사하강으로 자유에너지를 최소화한다. 다른 학습 알고리즘은 변경하지 않는다. 수식 (44)에서 첫째 항을 빼고 직접 영향만 쓰면 tPC(BP)가 된다.

$$F^{(t)} = \frac{1}{2H} \| x^{(t)} - \mu_x^{(t)} \|^2 + \frac{1}{2C} \| y^{(t)} - (W_l \phi(W_r x^{(t)} + b_r) + b_l) \|^2 \tag{47}$$

3.3 GRU

GRU는 LSTM보다 구조가 단순하지만 성능에서는 차이가 거의없다[14]. 그리고 순환부 이외의 구조를 tPC-RTRL과 동일하게 설정할 수 있어서, 순환부와 학습 방식의 차이를 비교하기 용이하다. 다만 GRU는 게이트에 직전 순환 상태가 행렬 곱으로 들어가 영향 행렬을 원소별로 줄일 수 없으므로, TBPTT[13]으로 학습한다. GRU 순환부의 구조는 다음과 같다.

$$r^{(t)} = \sigma(W_r u^{(t-1)} + U_r x^{(t-1)} + b_r) \tag{48}$$

$$g^{(t)} = \sigma(W_g u^{(t-1)} + U_g x^{(t-1)} + b_g) \tag{49}$$

$$\tilde{x}^{(t)} = \tanh(W_h u^{(t-1)} + U_h (r^{(t)} \odot x^{(t-1)}) + b_h) \tag{50}$$

$$x^{(t)} = (1 - g^{(t)}) \odot x^{(t-1)} + g^{(t)} \odot \tilde{x}^{(t)} \tag{51}$$

손실 함수는 수식 (52)와 같다. TBPTT[13]를 이용한 기울기 수식은 (53)과 같다. 매 타임 스텝마다 수식 (53-54)에 따라 가중치를 업데이트하며 Adam을 사용한다.

$$l^{(t)} = \frac{1}{2C} \| y^{(t)} - (W_l \phi(W_o x^{(t)} + b_o) + b_l) \|^2 \tag{52}$$

$$\nabla_\theta l^{(t)} \approx \sum_{s=t-k+1}^{t} \frac{\partial l^{(s)}}{\partial x^{(s)}} \frac{\partial x^{(s)}}{\partial \theta} \tag{53}$$

$$\theta \leftarrow \theta - \alpha_{lr} \overline{\nabla}_\theta^{(k)} l^{(t)} \tag{54}$$

3.4 필터링

필터링 실험에서는 관측이 K 스텝마다 주어진다. 관측이 들어오면 가중치를 고정한 상태에서 추론을 통해 숨겨진 상태만 수정하여 자유에너지를 최소화하고, 그렇게 수렴한 숨겨진 상태를 이용하여 이후의 관측을 예측한다.

PC계열 모델(tPC, tPC-RTRL)은 3.1.2절의 추론을 그대로 사용하여, 수식 (30)을 x와 o에 대해 최소화한다. BP 계열 모델(tPC(BP), tPC-RTRL(BP))는 o를 순방향으로 고정한 수식 (47)을 x에 대해서만 경사하강으로 최소화한다.

GRU는 PC 기반 모델이 아니어서 최소화할 자유에너지와 추론 단계가 없다. 필터링을 하려면 별도의 알고리즘을 고안해야 하므로 실험에서 제외한다.

3.5 제어 신호 추론

현재 시점을 t, 제어 신호를 추론할 미래 구간의 길이를 h라 하자. 제어 신호 추론은 시점 t 이후 h 스텝의 목표 관측 열 $Y^* = (y^{*(t+1)}, ..., y^{*(t+h)})$이 주어졌을 때, 이를 가장 잘 설명하는 제어신호 열 $U = (u^{(t)}, ..., u^{(t+h-1)})$를 찾는 문제로 정의한다. 이때 가중치는 수정하지 않는다. 수식 (55) 로 나타낸 사후 확률을 구하는

것과 같다.

$$p(U|Y^*) = \frac{p(Y^*|U)p(U)}{p(Y^*)} \tag{55}$$

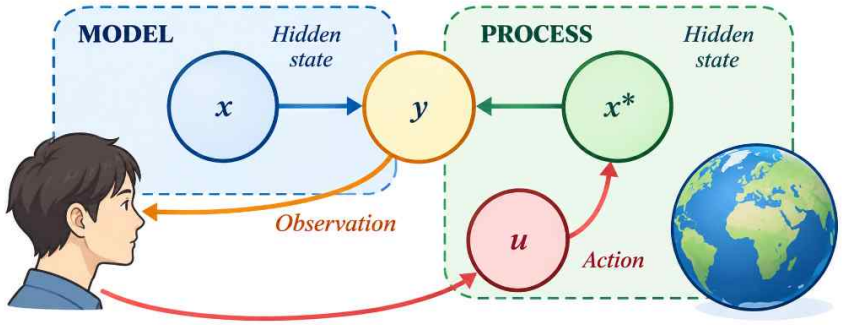


그림 5 생성 모델과 생성 과정. 본 연구의 제어 신호 추론은 생성 모델 안에서만 수행된다[16]

Fig. 5. Generative model and generative process; control inference in this study is performed only within the generative model[16]

그림 5는 생성 모델과 생성 과정을 구분하여 나타낸다[16]. 생성 과정은 실제 로봇의 동역학이다. 그 숨겨진 상태 x^*는 직접 관측할 수 없으며, 센서를 통한 관측 y만 얻을 수 있다.

본 논문에서 생성 모델은 이 과정을 근사하도록 학습한 tPC-RTRL이며, 제어 신호 u는 순환부의 입력(수식 29)으로 모델 안에도 존재한다. 본 연구의 제어 신호 추론은 생성 모델 안에서만 수행된다. 미래의 목표 관측을 아직 얻지 않은 관측처럼 취급하고, 이를 가장 잘 설명하는 u를 숨겨진 원인처럼 추론한다. 이는 관측을 바꾸는 행동이 아니라 u에 대한 믿음을 바꾸는 지각 쪽의 최소화이다.

이 방법은 일반화된 좌표와 기대 자유에너지를 이용하는 Friston의 능동 추론 구현과도 다르다[18]. 추론한 u를 실제 로봇에 인가하여 관측을 바꾸는 단계는 본 연구에서 다루지 않는다.

모델 증거 $p(Y^*)$는 U에 의존하지 않으므로 사후 확률을 최대화하는 제어 신호 열 U^*는 수식 (56)과 같이 음의 로그 결합 확률을 최소화 하는 U와 같다.

$$U^* = \arg\min_U [-\ln p(Y^*|U) - \ln p(U)] \tag{56}$$

이를 다시 정규분포로 가정한다. 모터 토크는 시간에 따라 연속적으로 변하므로, 사전 확률의 평균은 직전 시점의 실측 제어 신호 $u^{(t-1)}$로 둔다. 상수항을 생략하면 목적 함수는 수식 (57)과 같다.

t는 추론이 진행되는 동안 고정하고, 미래 시점은 t로 부터의 상대 스텝으로 나타낸다. $\tau(\tau = 1,\ldots,h)$는 오차를 계산하는 시점을 나타낸다. $s(s = 1,\ldots,h)$는 업데이트하려는 제어 신호의 시점을 나타낸다.

$$E(U) = \sum_{\tau=1}^{h} \frac{1}{2C} \| y^{*(t)} - \hat{y}^{(t+\tau)} \|^2 + \sum_{s=0}^{h-1} \frac{1}{2I} \| \hat{u}^{(t+s)} - u^{(t-1)} \|^2 \tag{57}$$

관측 예측 $\hat{y}^{(t+\tau)}$는 수식 (58)과 같이 순환 숨겨진 상태 $x^{(t)}$에서 출발해 모델을 전개하여 얻는다. 여기서 함수 f는 tPC-RTRL의 순환부 수식 (38-43)이나 GRU 순환부 수식 (48-51)이 될 수 있다.

$$\begin{aligned} &\mu_x^{(t)} = x^{(t)},\ \mu_x^{(t+\tau)} = f(\mu_x^{(t+\tau-1)}, u^{(t+\tau-1)}) \\ &\hat{y}^{(t+\tau)} = W_l \phi(W_o \mu_x^{(t+\tau)} + b_o) + b_l,\ \tau = 1,\ldots,h \end{aligned} \tag{58}$$

미래 관측은 주어지지 않으므로, 전개 과정에서 순환 숨겨진 상태와 중간 숨겨진 상태를 관측으로 보정하지 않고 예측값을 그대로 사용한다. 그러면 수식 (30)의 순환 숨겨진 상태 오차와 중간 숨겨진 상태 오차항이 0이 되어 관측 예측 오차항만 남는다.

제어 신호는 수식 (59)으로 갱신한다. 모든 제어신호의 초기값은 사전 평균 $u^{(t-1)}$로 두고, 정해진 횟수만큼 반복한 뒤, 첫 제어신호 $u^{(t)}$를 추론 결과로 사용한다.

$$u^{(t+s)} \leftarrow u^{(t+s)} - \alpha_{ir} \frac{\partial E(U)}{\partial u^{(t+s)}} \tag{59}$$

수식 (57)에서 관측 예측과 중간 숨겨진 상태는 제어 신호를 직접 인자로 갖지 않는다. 제어신호 $u^{(t)}$는 순환부를 통해서만 $\mu_x^{(t+s+1)}$에 영향을 주고, $\mu_x^{(t+s+1)}$는 이후의 모든 상태를 거쳐 $\hat{y}^{(t+s+1)}, \ldots, \hat{y}^{(t+h)}$에 영향을 준다. 이 의존 관계는 수식 (60)과 같다.

$$u^{(t+s)} \rightarrow \mu_x^{(t+s+1)} \rightarrow \mu_x^{(t+s+2)} \rightarrow \cdots \rightarrow \mu_x^{(t+h)},\ \mu_x^{(t+\tau)} \rightarrow \hat{y}^{(t+\tau)} \tag{60}$$

각 시점의 관측 예측 오차가 같은 시점의 순환 숨겨진 상태에 주는 직접 기울기를 $e^{(t+\tau)}$라 하면 수식 (61)과 같다. 중간 숨겨진 상태와 최종 예측 구조가 같으므로 이 식은 tPC-RTRL과 GRU에 공통이다.

$$e^{(t+\tau)} = -\frac{1}{C} W_o^T((W_l^T(y^{*(t+\tau)} - \hat{y}^{(t+\tau)})) \odot m_r^{(t+\tau)}) \tag{61}$$

$$m_r^{(t)} = \begin{cases} 1 \text{ if } W_r \mu_x^{(t+\tau)} + b_r > 0 \\ 0 \; otherwise \end{cases} \tag{62}$$

수식 (57)의 제어신호에 대한 기울기는 수식 (63)과 같다. 제어 신호 사전 확률 항에서 $u^{(t+s)}$를 포함하는 것은 하나뿐이다.

$$\frac{\partial E(U)}{\partial u^{(t+s)}} = \sum_{\tau=s+1}^{h} \left(\frac{\partial \mu_x^{(t+\tau)}}{\partial u^{(t+s)}}\right)^T e^{(t+\tau)} + \frac{1}{I}(u^{(t+s)} - u^{(t-1)}) \tag{63}$$

수식 (63)의 첫 번째 항에 있는 순환 숨겨진 상태를 제어 신호로 미분하는 것은 수식 (64)과 같이 미분 연쇄 법칙으로 이용할 수 있다.

$$\begin{aligned}
&\frac{\partial \mu_x^{(t+1)}}{\partial \mu_x^{(t)}} \doteq J_x^{(t)} \\
&\frac{\partial \mu_x^{(0)}}{\partial u^{(0)}} \doteq J_u^{(0)} \\
&\frac{\partial \mu_x^{(1)}}{\partial u^{(0)}} = \frac{\partial \mu_x^{(1)}}{\partial \mu_x^{(0)}} \frac{\partial \mu_x^{(0)}}{\partial u^{(0)}} = J_x^{(1)} J_u^{(0)} \\
&\frac{\partial \mu_x^{(2)}}{\partial u^{(0)}} = \frac{\partial \mu_x^{(2)}}{\partial \mu_x^{(1)}} \frac{\partial \mu_x^{(1)}}{\partial \mu^{(0)}} \frac{\partial \mu_x^{(0)}}{\partial u^{(0)}} = J_x^{(2)} J_x^{(1)} J_u^{(0)}
\end{aligned} \tag{64}$$

이를 수식 (63)에 적용하면 수식 (65) 처럼 되는데 이때 패턴을 볼 수 있다.

$$\begin{aligned}
&\frac{\partial E(U)}{\partial u^{(t+s)}} \\
&= J_u^{(0)}(e^{(0)} + J_x^{(1)}(e^{(1)} + J_x^{(2)} e^{(2)})) + \cdots + \frac{1}{I}(u^{(t+s)} - u^{(t-1)})
\end{aligned} \tag{65}$$

이 패턴을 수식 (66)을 이용하여 단순화하면 수식 (67)과 같이 간단해진다. 수식 (66-67)의 J_x와 J_u의 값은 tPC-RTRL일 때와

GRU일때 각각 다르다.

$$z^{(t+h)} = e^{(t+h)} \tag{66}$$
$$z^{(t+\tau)} = e^{(t+\tau)} + J_x^{(t+\tau+1)} z^{(t+\tau+1)}$$

$$\frac{\partial E(U)}{\partial u^{(t+s)}} = J_u^{(t+s+1)} z^{(t+s+1)} + \frac{1}{I}(u^{(t+s)} - u^{(t-1)}) \tag{67}$$

3.5.1 tPC-RTRL의 제어 신호 추론

J_u를 구하기 위해서 수식 (38-43)의 $u^{(t)}$에 대한 미분이 필요하며 수식 (68-71)과 같다.

$$\frac{\partial g_z^{(t)}}{\partial u^{(t)}} = g_z^{(t)}(1 - g_z^{(t)})\, W_z \tag{68}$$
$$\alpha_z \doteq g_z^{(t)}(1 - g_z^{(t)})$$
$$\frac{\partial g_z^{(t)}}{\partial u^{(t)}} = \alpha_z W_z$$

$$\frac{\partial p^{(t)}}{\partial u^{(t)}} = W_{in} \tag{69}$$

$$\frac{\partial a^{(t)}}{\partial u^{(t)}} = \frac{\partial a^{(t)}}{\partial g_a^{(t)}} \frac{\partial g_a^{(t)}}{\partial u^{(t)}} = (c log(\Lambda) a^{(t)})(g_a^{(t)}(1 - g_a^{(t)})\, W_a) \tag{70}$$
$$\alpha_a \doteq c \log(\Lambda) a^{(t)} g_a^{(t)}(1 - g_a^{(t)})$$
$$\frac{\partial a^{(t)}}{\partial u^{(t)}} = \alpha_a W_a$$

$$\frac{\partial \gamma^{(t)}}{\partial u^{(t)}} = \frac{\partial \gamma^{(t)}}{\partial a^{(t)}} \frac{\partial a^{(t)}}{\partial u^{(t)}} = -\frac{a^{(t)}}{\gamma^{(t)}} c log(\Lambda) a^{(t)} g_a^{(t)}(1 - g_a^{(t)})\, W_a \tag{71}$$
$$= -\frac{a^{(t)}}{\gamma^{(t)}} \alpha_a W_a$$

수식 (68-71)을 이용하면 수식 (43)의 $u^{(t)}$에 대한 미분을 수식 (72)와 같이 구할 수 있으며, 수식 (43)로부터 J_x 수식 (73)을 구할 수 있다.

$$\begin{aligned}
&\frac{\partial \mu_x^{(t)}}{\partial u^{(t)}} \\
&= \bar{x}^{(t-1)} \alpha_a W_a + g_z^{(t)} p^{(t)} (-\frac{a^{(t)}}{\gamma^{(t)}} \alpha_a) W_a + \gamma^{(t)} p^{(t)} \alpha_z W_z + \gamma^{(t)} g_z^{(t)} W_{in} \\
&= \alpha_a (\bar{x}^{(t-1)} - \frac{g_z^{(t)} p^{(t)} a^{(t)}}{\gamma^{(t)}}) W_a + \gamma^{(t)} p^{(t)} \alpha_z W_z + \gamma^{(t)} g_z^{(t)} W_{in} \\
&= J_u^{(t)}
\end{aligned} \tag{72}$$

$$J_x^{(t)} = \frac{\partial \mu_x^{(t+1)}}{\partial \mu_x^{(t)}} = a^{(t)} \tag{73}$$

3.5.2 GRU의 제어 신호 추론

GRU에서 J_u를 계산하기 위해 수식 (48-51)의 $u^{(t)}$에 대한 미분이 필요하며 이는 수식 (74-76)과 같다. GRU의 J_u 는 수식 (77)과 같다.

$$\frac{\partial g^{(t)}}{\partial u^{(t-1)}} = g^{(t)} (1 - g^{(t)}) W_g \tag{74}$$

$$\frac{\partial r^{(t)}}{\partial u^{(t-1)}} = r^{(t)} (1 - r^{(t)}) W_r \tag{75}$$

$$\frac{\partial \bar{x}^{(t)}}{\partial u^{(t-1)}} = (1 - (\bar{x}^{(t)})^2)(W_h + U_h x^{(t-1)} \frac{\partial r^{(t)}}{\partial u^{(t-1)}}) \tag{76}$$

$$J_u^{(t)} = \frac{\partial x^{(t)}}{\partial u^{(t-1)}} = -x^{(t-1)}\frac{\partial g^{(t)}}{\partial u^{(t-1)}} + \tilde{x}^{(t)}\frac{\partial g^{(t)}}{\partial u^{(t-1)}} + g^{(t)}\frac{\partial \tilde{x}^{(t)}}{\partial u^{(t-1)}} \quad (77)$$
$$= \frac{\partial g^{(t)}}{\partial u^{(t-1)}}(\tilde{x}^{(t)} - x^{(t-1)}) + g^{(t)}\frac{\partial \tilde{x}^{(t)}}{\partial u^{(t-1)}}$$
$$= (\tilde{x}^{(t)} - x^{(t-1)})g^{(t)}(1-g^{(t)})W_g$$
$$+ g^{(t)}(1-(\tilde{x}^{(t)})^2)(W_h + U_h x^{(t-1)} r^{(t)}(1-r^{(t)})W_r)$$

J_x를 구하기 위해서 다음과 같이 수식 (78-85)을 이용한다.

$$\frac{\partial x^{(t)}}{\partial x^{(t-1)}} = (1-g^{(t)}) \quad (78)$$

$$\frac{\partial x^{(t)}}{\partial g^{(t)}} = \tilde{x}^{(t)} - x^{(t-1)} \quad (79)$$

$$\frac{\partial g^{(t)}}{\partial x^{(t-1)}} = g^{(t)}(1-g^{(t)})U_g \quad (80)$$

$$\frac{\partial x^{(t)}}{\partial \tilde{x}^{(t)}} = g^{(t)} \quad (81)$$

$$\frac{d\tilde{x}^{(t)}}{dx^{(t-1)}} = (1-(\tilde{x}^{(t)})^2)U_h r^{(t)} \quad (82)$$

$$\frac{\partial \tilde{x}^{(t)}}{\partial r^{(t)}} = (1-(\tilde{x}^{(t)})^2)U_h x^{(t-1)} \quad (83)$$

$$\frac{\partial r^{(t)}}{\partial x^{(t-1)}} = r^{(t)}(1-r^{(t)})U_r \quad (84)$$

$$\frac{d\tilde{x}^{(t)}}{dx^{(t-1)}} = \frac{\partial \tilde{x}^{(t)}}{\partial x^{(t-1)}} + \frac{\partial \tilde{x}^{(t)}}{\partial r^{(t)}}\frac{\partial r^{(t)}}{\partial x^{(t-1)}} \quad (85)$$

GRU의 J_u는 수식 (86)과 같다.

$$
\begin{aligned}
J_x^{(t)} &= \frac{\partial x^{(t)}}{\partial x^{(t-1)}} = \frac{\partial x^{(t)}}{\partial x^{(t-1)}} + \frac{\partial x^{(t)}}{\partial g^{(t)}} \frac{\partial g^{(t)}}{\partial x^{(t-1)}} + \frac{\partial x^{(t)}}{\partial \tilde{x}^{(t)}} \frac{\partial \tilde{x}^{(t)}}{\partial x^{(t-1)}} \\
&= \frac{\partial x^{(t)}}{\partial x^{(t-1)}} + \frac{\partial x^{(t)}}{\partial g^{(t)}} \frac{\partial g^{(t)}}{\partial x^{(t-1)}} + \frac{\partial x^{(t)}}{\partial \tilde{x}^{(t)}} \left(\frac{d\tilde{x}^{(t)}}{dx^{(t-1)}} + \frac{\partial \tilde{x}^{(t)}}{\partial r^{(t)}} \frac{\partial r^{(t)}}{\partial x^{(t-1)}} \right) \\
&= (1 - g^{(t)}) \\
&+ (\tilde{x}^{(t)} - x^{(t-1)}) g^{(t)} (1 - g^{(t)}) U_g \\
&+ g^{(t)} (1 - (\bar{x}^{(t)})^2) U_h r^{(t)} (1 + x^{(t-1)} (1 - r^{(t)}) U_r)
\end{aligned}
\tag{86}
$$

Ⅳ. 실험 및 결과 분석

4.1 데이터세트

실험에는 Industrial Robot Identification Benchmark 데이터세트[11, 12]를 사용한다. 이 데이터세트는 6축 산업용 로봇 KUKA KR300 R2500 ultra SE를 구동하며 수집되었다. 각 궤적은 길이 60.6초이며, 모든 궤적은 같은 원점 자세에서 시작하고 끝난다. 36개의 서로 다른 궤적을 각각 두 번 실행하여 총 72개 궤적, 약 73분 분량의 데이터가 있다[11].

벤치마크는 데이터를 두 가지 형태로 제공한다[11, 12]. 첫째는 가공하지 않은 raw data이다. raw data는 250Hz로 기록되어 있으며, 관절 위치와 관절 속도, 모터 토크 등의 측정값이 들어 있다. 둘째는 식별 과제용으로 가공한 10Hz 데이터이다. 순방향 모델 데이터는 모터 토크 6채널을 입력으로, 관절 위치 6채널을 출력으로 한다. 역모델 데이터는 관절 위치·속도·가속도 18채널을 입력으로, 모터 토크 6채널을 출력으로 한다. 두 데이터 모두 12개 원자료 파일 중 마지막 파일을 테스트(3,636 스텝)에, 나머지 11개 파일을 학습(39,988 스텝)에 사용한다.

4.1.1 가공 데이터를 사용하지 않은 이유

본 연구는 실시간 학습 및 예측, 필터링, 제어 신호 추론을 하나의 생성 모델로 수행한다. 따라서 세 과제는 같은 관측 데이터 y와 같은

제어 신호 u를 사용해야 한다. 또한 로봇의 상태는 6개 관절의 위치와 속도로 이루어지므로[11], 본 논문의 실험에는 관측 데이터에 위치와 속도를 모두 포함하였다. 그러나 제공된 순방향 모델 데이터에는 속도가 없다. 역모델 데이터에는 속도와 가속도가 있으나, 이는 측정값이 아니라 위치 신호로부터 추정한 값이다[12]. 반면 raw data에는 모터 리졸버로 측정한 관절 속도가 위치와 함께 기록되어 있다. 이에 본 연구는 추정값 대신 측정된 속도를 사용하기 위해, raw data로부터 관측 데이터와 제어 신호를 직접 구성하였다.

4.1.2 관측 데이터와 제어 신호

관측 데이터 y는 관절 위치 6채널[deg]과 관절 속도 6채널[deg/s], 총 C = 12채널이다. 위치와 속도는 모두 모터 축에 설치된 리졸버의 측정값을 사용하였다. 제어 신호 u는 측정된 모터 토크 6채널[Nm]이며, I = 6이다. 이 토크에는 순방향 토크와 역방향 제어기 토크가 모두 포함된다[11]. 모델에는 토크만 입력하고 과거 관측은 입력하지 않는다. 이는 벤치마크가 정의한 순방향 모델 과제와 같다[11].

raw data는 12개 파일로 나뉘어 있고, 파일마다 서로 다른 궤적 3개가 연속으로 두 번씩 들어 있다. 파일들은 기록 시각 순서로 번호를 매겼다.

4.1.3 데시메이션

raw data는 1초에 250번, 즉 4ms 간격으로 기록되어 있다(샘플링 주파수 250Hz). 그러나 이 데이터세트의 궤적은 관절 위치의 주파수 성분이 0.3Hz 이하가 되도록 설계되었다[11]. 이를 표현하는데 4ms

간격은 지나치게 촘촘하다. 또한 PC 학습은 매 샘플마다 추론과 가중치 업데이트를 수행하므로, 샘플이 많을수록 계산량이 커진다. 따라서 샘플을 일정 간격으로 솎아내어 샘플링 주파수를 낮추었다. 이를 데시메이션(decimation)이라 한다[20]. 본 연구는 25개 샘플 중 1개만 남겨 샘플링 주파수를 10Hz, 즉 100ms 간격으로 낮추었다. 이는 벤치마크의 가공 데이터와 같은 샘플링 주파수다[11].

4.1.4 앨리어싱과 저역통과 필터

샘플을 단순히 솎아내기만 하면 앨리어싱(aliasing)이라는 문제가 생긴다[20]. 샘플링 주파수가 f_s이면, f_s/2보다 빠르게 변하는 성분은 표현할 수 없다. 이 한계를 나이퀴스트 주파수라 한다[20]. 10Hz로 샘플링하면 나이퀴스트 주파수는 5Hz이다. 이보다 빠른 성분은 사라지지 않고, 느린 가짜 성분으로 바뀌어 데이터에 남는다.

앨리어싱을 막으려면 솎아내기 전에 나이퀴스트 주파수보다 빠른 성분을 미리 제거해야 한다. 이를 위해 저역통과 필터(low-pass filter)를 사용한다[20]. 저역통과 필터는 차단 주파수보다 느린 성분은 통과시키고 빠른 성분은 제거한다. 본 연구에서는 차단 주파수를 약 5Hz로 두었다. 이렇게 데시메이션 앞에서 앨리어싱을 막는 필터를 안티앨리어싱 필터라고 한다. 실험에 사용한 저역통과 필터는 FIR(Finite Impulse Response)필터이다. 필터를 사용하면 필터를 통과한 데이터의 파형이 입력보다 늦게 나타나는 시간 지연이 발생하는데, 이를 위해 특정 시점의 출력을 계산할 때 미래 샘플을 이용하는 영위상(zero-phase)처리를 하였다[20].

4.1.5 정규화

모든 채널은 채널별 평균과 표준편차로 정규화하였다. 학습을 시작할 때 이후에 들어올 데이터를 알 수 없다고 가정하여 평균과 표준편차는 워밍업에 쓰는 파일 0~2에서만 구하고, 같은 값을 모든 파일에 적용하였다. 또한 전체 데이터로 정규화하면, 시간에 따라 마찰 특성이 변하면서 생기는 데이터의 변화가 정규화 값에 미리 반영된다. 그러면 모델이 적응해야 할 변화의 일부를 정규화가 대신 보정하게 된다.

이상과 같이 본 연구의 데이터는 관측 신호, 센서, 전처리, 테스트 구간이 벤치마크의 가공 데이터와 다르다. 따라서 4장의 결과는 벤치마크 문서의 선형 베이스라인 수치[11]와 직접 비교할 수 없다.

4.2 실험 절차와 초매개변수

4.2.1 실험 절차

모든 실험은 파일 0~2번 파일로 5번 반복 학습시켜 워밍업을 한다. 실시간 학습 및 예측 실험의 경우 파일 3~11을 시간 순서대로 한 번씩 입력하며 매 스텝 예측 후 갱신하는 실시간 학습을 시뮬레이션했다. 스텝당 연산 시간은 가장 짧은 tPC-RTRL(BP)는 0.5596ms, 가장 긴 k=50일때의 GRU는 10.1989ms로 샘플 주기(100ms)보다 짧아 실시간 조건을 만족한다. 단, 데이터 세트는 FIR 필터로 영위상 전처리한 것이다.

필터링과 제어신호 추론 실험에서는 0번에서 10번 까지의 파일을 온라인 학습시킨 후 11번 파일에서만 MSE를 측정하여 평균을 구한다. 이러한 방식으로 총 10회 실험하여 그 평균과 표준편차를 측정한다.

표 3은 파일 인덱스와 실험 구간을 나타낸다.

Table 3 File index and experiments section
표 3 파일 인덱스와 실험 구간

Experiment	File index	Use
Warm up	0 ~ 2	Model weights stablize, 5 repeat
Real-Time learning & prediction	3 ~ 11	Real-Time learning & prediction
Filtering	0 ~ 10	Online Learning
	11	Filtering & Prediction
Control inference	0 ~ 10	Online Learning
	11	Control Inference

4.2.2 초매개변수 설정

순환 숨겨진 상태 차원 H=128, 중간 숨겨진 상태 차원 R=128, 관측 데이터 차원 C=12, 제어 신호 차원 I=6이며 실시간 학습에서 추론 반복 횟수는 3, 추론 크기는 1, 모멘텀은 0.9이다. 타임 스텝당 학습은 한번만 하고, Adam 옵티마이저를 사용하며 $\beta 1 = 0.9$, $\beta 2 = 0.999$, 학습률은 5e-5이다. 필터링 간격은 $K \in \{1,10,25,100\}$을 사용하였으며 이때 추론 반복 횟수는 100회다. 제어 신호 추론에서 $h \in \{1,2,4,8,16\}$스텝 만큼 미래의 목표를 대상으로 제어 신호를 추론한다. 이때 추론 스텝은 5회, 추론률은 0.1이다. GRU 모델의 TBPTT 윈도우 크기는 $k \in \{1,5,10,25,50\}$을 사용하였으며 윈도우는 1칸씩 슬라이드한다. 각 실험을 다른 시드별로 10번하여 평균과 표본 표준편차를 구하였다.

tPC-RTRL과 tPC-RTRL(BP) 기울기의 코사인 유사도와 크기를

비교했다. 파일 0번의 최초 3,000스텝을 사용하였으며, Adam을 적용하기 전의 기울기를 이용하였다. 다른 시드별로 3회 실행하여 평균을 내었다. 그림 6은 그 비교 결과이다.

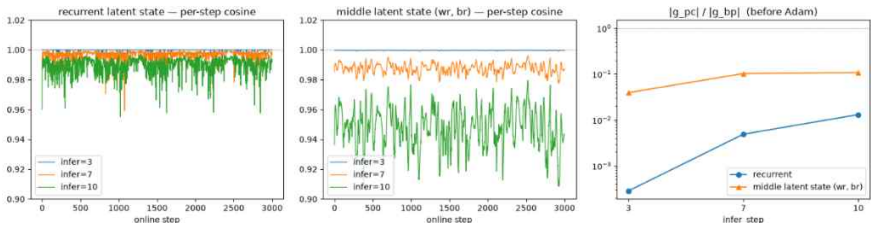


그림 6. tPC-RTRL과 tPC-RTRL(BP)의 기울기 코사인 유사도와 크기 비교

Fig. 6. Comparison of gradient cosine similarity and magnitude between tPC-RTRL and tPC-RTRL(BP)

추론 스텝 3에서 순환 잠재 상태, 중간 잠재 상태 가중치의 기울기 모두 코사인 유사도가 0.999이다. 그러나 그 크기 비율은 중간 숨겨진 상태에서 약 0.04, 순환 숨겨진 상태에서 약 0.0003이다. tPC-RTRL은 tPC-RTRL(BP)의 근사 등가 영역에 있으며 완전 수렴을 강제하지 않고 실험한다.

4.3 실시간 학습 및 예측 성능

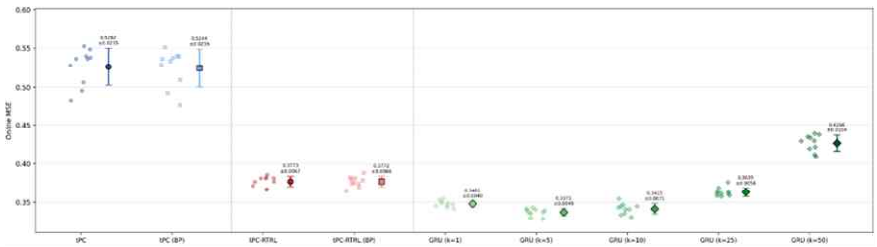


그림 7. 실시간 학습 및 예측 실험 결과

Fig. 7. Experiment result of Real-time learning and prediction

그림 7은 실시간 학습 및 예측 실험 결과이다. tPC와 tPC(BP)에 영향 행렬을 추가한 tPC-RTRL, tPC-RTRL(BP)의 MSE가 각각 39.46%(0.1489), 39.02%(0.1472)만큼 더 작다. tPC-RTRL보다 tPC-RTRL(BP)의 MSE가 0.03%(0.0001)만큼 큰데 이는 각 표준편차보다도 작다. 오프라인 배치 학습[7]과 마찬가지로 차이가 거의 없다. GRU의 성능은 윈도우 길이 k에 따라 성능이 달랐다. k가 5일때 MSE가 가장 낮았고, k가 50일때 가장 크다. 윈도우 길이 별로 적합한 초매개변수가 있기 때문이라고 추정된다. 이에 대한 자세한 분석은 본 논문의 목적이 아니기에 하지 않는다. k가 5일때 GRU 모델과 비교하면 tPC-RTRL 실시간 학습 및 예측 MSE가 약 11.89% 더 크다.

4.4 필터링 성능

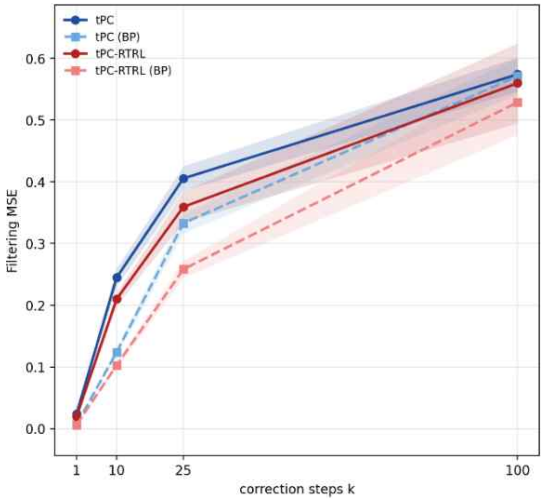


그림 8 필터링 실험 결과

Fig. 8. Filtering experiment result

그림 8은 필터링 실험 결과이다. BP계열의 MSE가 PC계열의 MSE보다 눈에 띄게 낮다. 특히 K가 10일때는 tPC의 MSE가 tPC(BP)보다 97.21%(0.12074)만큼 더 크며, tPC-RTRL의 MSE가 tPC-RTRL(BP)보다 104.15%(0.1072)만큼 더 크다. K가 25일때는 tPC의 MSE가 tPC(BP)보다 21.78%(0.072)만큼 더 크고, tPC-RTRL의 MSE가 tPC-RTRL(BP)보다 39.07%(0.10096)만큼 더 크다. 이렇게 차이가 나는 이유는 BP 계열의 경우 중간 숨겨진 상태를 사용하지 않기 때문이다.

PC 계열의 경우 3.1.2 추론을 통해 숨겨진 상태 x와 o를 함께 이용하여 자유에너지를 최소화한다. 그러나 예측에서는 수렴된 o를 사용하여 예측하지 않고, 수렴된 x를 사용하여 μ_x를 예측하고 이를 이용하여 o를 예측하고 예측한 o를 이용하여 관측 데이터를 예측

방식을 사용한다. 그러니까 추론 단계에서 수렴한 o는 예측 단계에서 사용하지 않게 된다.

반면에 BP 계열의 경우 수식 (47)의 자유에너지를 보면 중간 잠재변수 o가 존재하지 않고 x의 변화량만 구한다. 따라서 예측 단계에서 필터링 성능이 더 우수하다.

4.5 제어 신호 추론 성능

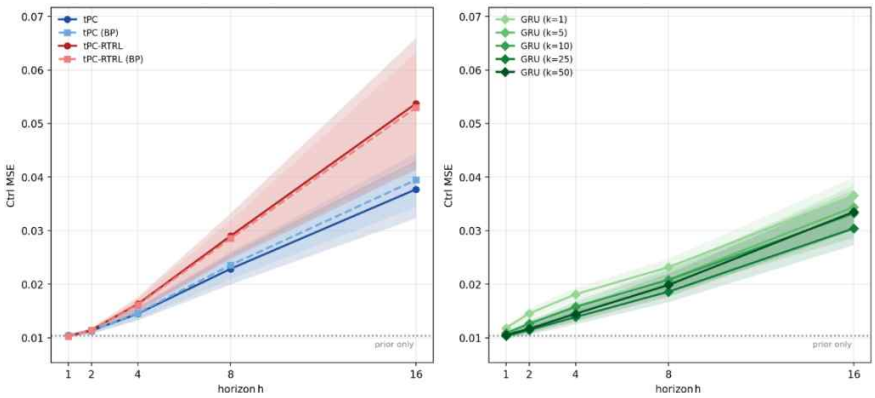


그림 9 제어 신호 추론 실험 결과

Fig. 9. Control signal inference experiment result

그림 9는 제어 신호 추론 실험 결과이다. tPC-RTRL 계열과 tPC 계열의 차이가 뚜렷하다. 직전 제어신호 만으로 MSE를 측정하면 0.01038이다. 그림 9에 가로 점선으로 이를 표현했다. 모든 모델이 h가 2만 되어도 이를 넘어선다. 그렇다면 어느 모델이 직전 제어 신호를 더 적게 수정하는지가 중요해 지는데 tPC-RTRL이 장기 학습을 더 잘했기 때문에 처음 보는 데이터의 직전 제어 신호를 많이 수정하고 따라서 tPC계열보다 MSE가 크게 나온것으로 추정된다. 이번에도

PC와 BP의 성능 차이는 거의 없다. tPC-RTRL에 비해 GRU의 MSE가 낮은 것은 GRU의 게이트 수식 (48-51)에 보면 직전 순환 숨겨진 상태가 완전 연결 되어있기 때문이라고 추정된다. 제어 신호 추론에서 모델별 성능 차이가 무엇 때문인지 더 면밀한 분석이 필요하다.

4.6 비용

RG-LRU와 GRU의 메모리 사용량을 비교한다. 중간 숨겨진 상태와 최종 예측 구조는 두 모델이 동일하므로 순환부만 비교한다.

tPC-RTRL의 RG-LRU 학습에서 메모리는 모델 가중치(θ_{rec}), Adam(m,v)의 모멘트 ($2\theta_{rec}$), 영향행렬에 필요한 메모리(θ_{rec}), 그리고 순환 잠재 상태 크기(H)를 합한 값이다. float 64(b = 8 bytes), 1KB = 1024 bytes, H = 128, I = 6 기준으로 계산한다. 수식 (38-43)의 가중치와 편향의 행렬 크기를 이용하여 순환부 메모리 크기를 알 수 있는데 이는 수식 (78)과 같다. 그리고 이를 이용하여 메모리 크기를 계산하면 수식 (79)과 같다.

$$Memory(\theta_{rec}) = 3HI + 4H \tag{78}$$

$$Memory_{RG-LRU} = (4Memory(\theta_{rec}) + H)b = 89KB \tag{79}$$

GRU 학습에서 메모리는 모델 가중치(θ_{GRU}), Adam(m,v)의 모멘트($2\theta_{GRU}$), TBPTT 학습을 위한 메모리가 필요하다. TBPTT 학습을 위해서는 윈도우 내 스텝마다 직전 순환 상태 $x^{(t-1)}$(H), 제어 입력 $u^{(t)}$(I), 게이트 계산 결과 $g^{(t)}$, $r^{(t)}$, $r^{(t)} \odot x^{(t-1)}$, $\mu^{(t)}$, $x^{(t)}$(5H)으로 총 6H+I 만큼의 메모리가 필요하다. 이들은 수식

(48-51)의 미분을 계산하기 위해 필요하며 재계산을 피하기 위해 $r^{(t)} \odot x^{(t-1)}$, $x^{(t)}$도 저장했다. GRU 순환부 가중치 메모리는 수식 (80)와 같다. 총 필요 메모리는 수식 (81)와 같다.

$$Memory(\theta_{GRU}) = 3(HI + H^2 + H) \tag{80}$$

$$\begin{aligned} Memory_{GRU} &= (3Memory(\theta_{GRU}) + H + k(6H + I))b \\ &\approx (1216 + 6.05k)KB \end{aligned} \tag{81}$$

순환부 가중치 메모리는 수식 (80)와 같다. 총 필요 메모리는 수식 (81)와 같다.

tPC-RTRL과 tPC-RTRL(BP)는 순환 잠재부의 메모리 크기가 동일하다. RG-LRU와 GRU의 메모리를 도표로 나타내면 그림 10과 같으며, 영향 행렬과 TBPTT학습 버퍼 메모리 비교를 도표로 나타내면 그림 11와 같다.

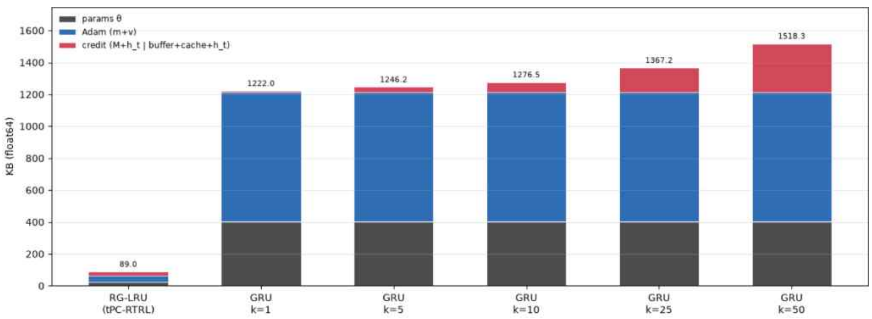


그림 10 RG-LRU(tPC-RTRL)와 GRU의 메모리 크기 비교

Fig. 10 Memory size comparison between RG-LRU(tPC-RTRL) and GRU

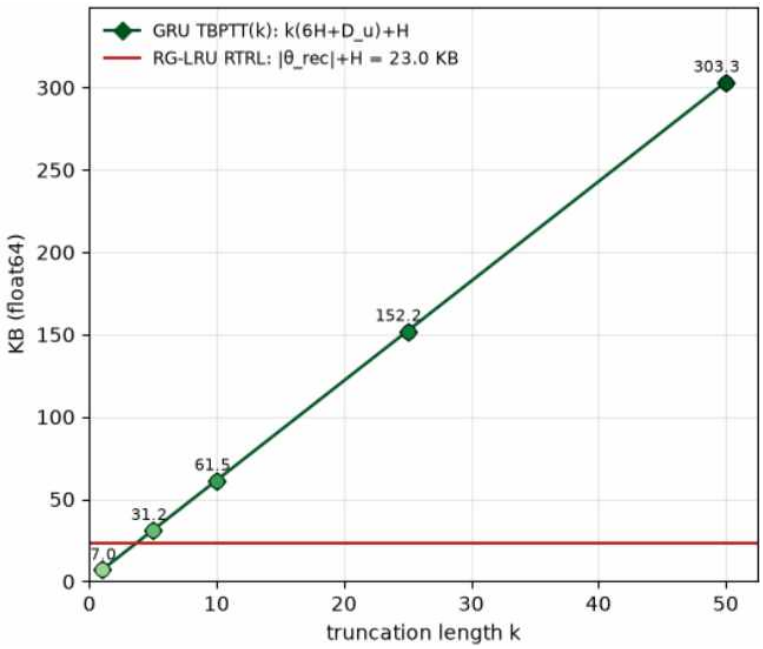


그림 11 영향행렬과 TBPTT 학습 버퍼 메모리 비교

Fig. 11 Memory size comparison between influence matrix and TBPTT learning buffer

타임 스텝당 학습 시간은 200 스텝 워밍업 이후 2,000 스텝 동안의 실행시간으로 측정하였다. 측정 시간에는 매 타임 스텝의 추론, 예측, 가중치 업데이트가 모두 포함된다. 측정 중에는 가비지 컬렉션을 비활성화 하여 측정 잡음을 줄였다. 시드를 다르게 하여 10회 실행 후 평균을 구하였다. 실험 환경은 Intel(R) Core(TM) i9-10900X CPU @ 3.70GHz, Python 3.13.15, Numpy 2.5.3 이다. 측정 결과는 그림 12와 같다.

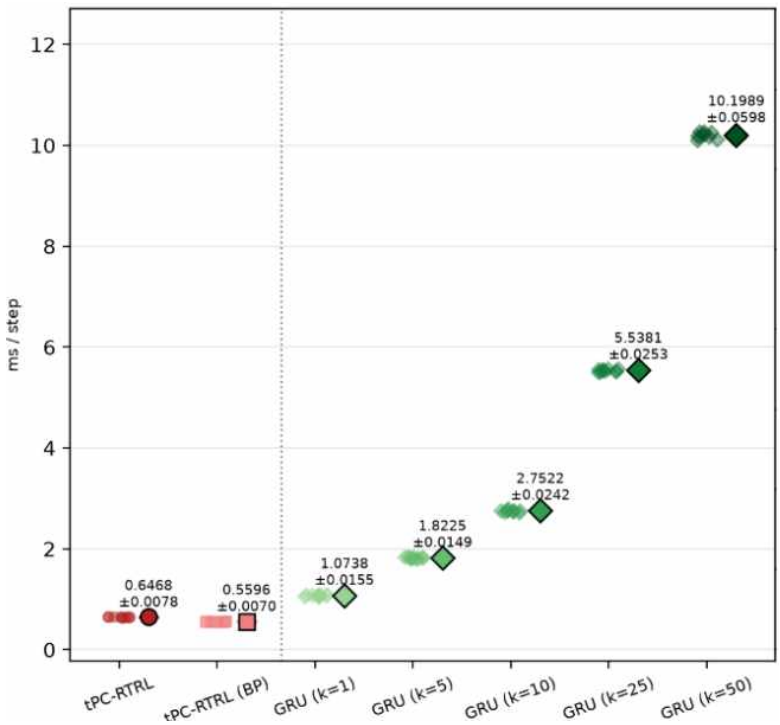


그림 12 실행 시간 측정 결과

Fig. 12 Execution time measurement results

tPC-RTRL의 경우 추론을 3회 실행하기 때문에 tPC-RTRL(BP)보다 실행 시간이 더 걸린다. GRU의 경우 윈도우 크기가 커질수록 실행 시간이 급격히 커진다.

k=5일때 GRU와 대비하여 tPC-RTRL 실시간 학습 및 예측 MSE가 약 11.89% 더 크다. 하지만 메모리 사용량은 tPC-RTRL이 약 92.86% 더 적으며, 실행 시간 또한 약 64.51 % 더 적다.

Ⅴ. 결론

tPC-RTRL에서 추론이 완전 수렴 한다면 BPTT와 등가이다. 하지만 tPC-RTRL의 추론을 완전 수렴 시키지 않은 근사 등가 영역에서 Industrial Robot benchmark를 이용한 실시간 학습 및 예측 성능은 tPC-RTRL(BP) 방식과 거의 동일하다. 반면 필터링 성능은 차이가 큰데, 그 이유는 tPC-RTRL의 경우 추론 과정에서 수렴한 중간 숨겨진 상태가 예측 단계에서는 사용되지 않기 때문이다. 제어 신호 추론 성능은 PC 계열과 BP 계열의 성능 차이는 거의 없다. 하지만 tPC-RTRL 계열 모델의 성능이 tPC 계열 모델의 성능 보다 더 나쁘다. GRU 모델의 경우 제어 신호 추론 성능이 가장 좋은데 그 이유는 게이트 구조상 직전 순환 숨겨진 상태가 완전 연결 형태를 가지기 때문이라고 추정된다. 제어 신호 추론에서의 모델별 성능 차이에 대해 더 면밀한 분석이 필요하다. 비용 측면에서 tPC-RTRL이 GRU 기반 모델보다 메모리와 실행 시간 관점에서 모두 우수하다. k=5일때 GRU 모델과 비교하면 tPC-RTRL 실시간 학습 및 예측 MSE가 약 11.89% 더 크지만, 메모리 사용량은 약 92.86% 더 적으며, 실행 시간 또한 약 64.51 % 더 적다.

5.1 향후 연구

능동 추론은 지각과 행동뿐 아니라 주의 집중과 정서가도 설명한다. 주의는 하위 계층 예측 오차의 정밀도를 조절하는 상위 상태로 정의되고[21], 정서가는 자유에너지의 시간에 따른 변화율로 정의되어 학습률을 조절한다[22]. 그러나 이러한 모델은 대부분 상태 수가 적은

단순한 과제에서만 시연되었다. 정밀도를 다루려면 자유에너지를 디랙 델타 분포로 가정하지 않고, 사후 불확실성도 추정해야 한다. 이를 라플라스 근사로 표현하려면 헤시안 행렬을 계산해야 하며, 이는 숨겨진 상태 차원의 제곱에 비례하는 메모리를 요구한다[23]. 헤시안 행렬 없이 사후분포를 직접 표본추출하는 방법도 제안되었으나[24], 이 역시 긴 표본추출 과정이 필요해 추론 시간이 길다. 따라서 이를 구현하여 실생활에 응용하기는 어렵다. 본 연구에서 tPC-RTRL은 GRU보다 적은 메모리와 실행 시간으로 실시간 학습이 가능함을 보였다. 향후에는 이 가벼운 구조 위에 정밀도 학습과 정서가 기반 학습률 조절을 더하여, 주의와 정서의 기능을 실시간 시스템에서 구현할 수 있는지 연구할 것이다.

참 고 문 헌

[1] Rumelhart, D., Hinton, G. & Williams, R. “Learning representations by back-propagating errors.” Nature 323, 533 – 536. Oct 1986. https://doi.org/10.1038/323533a0

[2] Crick, F. “The recent excitement about neural networks.” Nature 337, 129 – 132. Jan 1989. https://doi.org/10.1038/337129a0

[3] Lillicrap, T.P., Santoro, A., Marris, L. et al. “Backpropagation and the brain.” Nat Rev Neurosci 21, 335 – 346 April 2020. https://doi.org/10.1038/s41583-020-0277-3

[4] Whittington, J. C. R., Bogacz, R. “Theories of error back-propagation in the brain.” Trends Cogn. Sci. 23, 235 – 250 March 2019. https://doi.org/10.1016/j.tics.2018.12.005

[5] Whittington, J. C. R., Bogacz, R. “An Approximation of the Error Backpropagation Algorithm in a Predictive Coding Network with Local Hebbian Synaptic Plasticity.” Neural Comput. vol.29, no. 5, May 2017 1229 – 1262. https://doi.org/10.1162/NECO_a_00949

[6] P. J. Werbos, "Backpropagation through time: what it does and how to do it," in Proceedings of the IEEE, vol. 78, no. 10, pp. 1550-1560, Oct. 1990, https://doi.org/10.1109/5.58337.

[7] Potter, T., Rhodes, O. “Learning Long-Range Dependencies with Temporal Predictive Coding”. arXiv preprint. Jun 2026. https://doi.org/10.48550/arXiv.2602.18131

[8] Millidge, B., Tang, M., Osanlouy, M., Harper, NS., Bogacz, R. “Predictive coding networks for temporal prediction.” PLoS Comput Biol 20(4): e1011183. April, 2024. https://doi.org/10.1371/journal.pcbi.1011183

[9] Ronald. J. Williams, David Zipser.“A Learning Algorithm for Continually Running Fully Recurrent Neural Networks.” Neural Comput. June 1989; 1 (2): 270 – 280. doi: https://doi.org/10.1162/neco.1989.1.2.270

[10] De, S., Smith, S. L., Fernando, A., Botev, A., Cristian-Muraru, G., et al. “Griffin: Mixing gated linear recurrences with local attention for efficient language models.” arXiv preprint. https://doi.org/10.48550/arXiv.2402.19427

[11] J. Weigand, J. Götz, J. Ulmen, and M. Ruskowski, “Dataset and Baseline for an Industrial Robot Identification Benchmark,” Version 2.0, Workshop on Nonlinear System Identification Benchmarks, Leuven, Belgium, Apr. 2023. https://nbn-resolving.org/urn:nbn:de:hbz:386-kluedo-72843 [accessed: Oct. 22. 2026]

[12] J. Weigand, J. Götz, J. Ulmen, and M. Ruskowski, “Dataset and Baseline for an Industrial Robot Identification Benchmark,” TU Kaiserslautern, 2022, dataset. https://doi.org/10.26204/DATA/5

[13] R. J. Williams, J. Peng, “An efficient gradient-based algorithm for on-line training of recurrent network trajectories,” Neural Computation, vol. 2, no. 4, pp. 490 – 501, 1990, https://doi.org/10.1162/neco.1990.2.4.490.

[14] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” arXiv preprint. Dec 2014. https://doi.org/10.48550/arXiv.1412.3555.

[15] Friston, K. “The free-energy principle: a unified brain theory?.” Nat Rev Neurosci 11, 127 – 138. Jan 2010. https://doi.org/10.1038/nrn2787

[16] Parr, T., Pezzulo, G., Friston, K. J. “Active Inference: The Free Energy Principle in Mind, Brain, and Behavior.1:Overview” Cambridge, MA: MIT Press, 2022. https://doi.org/10.7551/mitpress/12441.003.0004

[17] Song, Yuhang et al. “Can the Brain Do Backpropagation? -Exact Implementation of Backpropagation in Predictive Coding Networks.” Advances in neural information processing systems vol. 33. 22566-22579. Apr 2020. https://papers.neurips.cc/paper_files/paper/2020/file/fec87a37cdeec1c6ecf8181c0aa2d3bf-Paper.pdf [accessed: Oct. 22. 2026]

[18] Parr, T., Pezzulo, G., Friston, K. J. “Active Inference: The Free Energy Principle in Mind, Brain, and Behavior.4: The Generative Models of Active Inference” Cambridge, MA: MIT Press, 2022. https://doi.org/10.7551/mitpress/12441.003.0007

[19] Rafal Bogacz. “A tutorial on the free-energy framework for modelling perception and learning”, Journal of Mathematical Psychology, Volume 76, Part B, Feb 2017, Pages 198-211, ISSN 0022-2496, https://doi.org/10.1016/j.jmp.2015.11.003.

[20] A. V. Oppenheim, R. W. Schafer, “Discrete-Time Signal Processing”, 3rd ed., Pearson, 2010. [accessed: Oct.27.2026]

[21] L. Sandved-Smith, C. Hesp, J. Mattout, K. Friston, A. Lutz, M. J. D. Ramstead, “Towards a computational phenomenology of mental action: modelling meta-awareness and attentional control with deep parametric active inference,” Neuroscience of Consciousness, vol. 7, no. 1, niab018, 2021. https://doi.org/10.1093/nc/niab018

[22] M. Joffily, G. Coricelli, “Emotional valence and the free-energy principle,” PLoS Comput Biol 9(6): e1003094, June 2013. https://doi.org/10.1371/journal.pcbi.1003094

[23] U. Zahid, Q. Guo, K. Friston, Z. Fountas, “Curvature-sensitive predictive coding with approximate Laplace Monte Carlo,” arXiv preprint, Mar 2023. https://doi.org/10.48550/arXiv.2303.04976

[24] G. Oliviers, R. Bogacz, A. Meulemans, “Learning probability distributions of sensory inputs with Monte Carlo predictive coding,” PLoS Comput Biol 20(10): e1012532, Oct 2024. https://doi.org/10.1371/journal.pcbi.1012532

Real-time Time Series Learing and Prediction Using tPC-RTRL

seokwon Lee

Department of Computer science & Enginnering Graduate School of Industrail Technology Gangwon national University(Gangneung)

Abstract

tPC-RTRL satisfies both spatial and temporal locality by combining tPC and RTRL based on an RG-LRU architecture. When inference fully converges, tPC-RTRL is mathematically equivalent to BPTT. In this paper, we investigate whether tPC-RTRL can achieve performance comparable to backpropagation (BP)-based learning in a real-time online learning environment, without enforcing full convergence of inference. We utilized the Industrial Robot Identification Benchmark dataset. Compared to tPC, tPC-RTRL reduced the prediction error (MSE) by approximately 39%, while showing a performance difference of only 0.03% relative to a BP-based variant. When compared to a GRU trained using TBPTT (k=5), tPC-RTRL exhibited an MSE that was 11.89% higher, yet it required 92.86% less memory and 64.51% less execution time.

□ keywords

Free-Energy Principle, Predictive Coding, Real-Time Recurrnt Learning, Online Learning, Time Series Prediction